





Testing Spring Boot Applications Demystified

A Hero's Journey Through the Spring Boot Testing Labyrinth

CoP Spring Boot @ REWE Group 25.03.2026

Philip Riecks - [PragmaTech GmbH](#) - [@rieckpil](#)

Participate During the Talk

Go to menti.com and use the code **4469 9652** to **anonymously** submit answers for the quizzes and add your questions during the talk.

Start with the **first three questions**:

- Which talk language do you prefer?
- Despite having LLMs and Code Agents, do you still write your tests by hand?
- Do You Enjoy Writing Automated Tests?

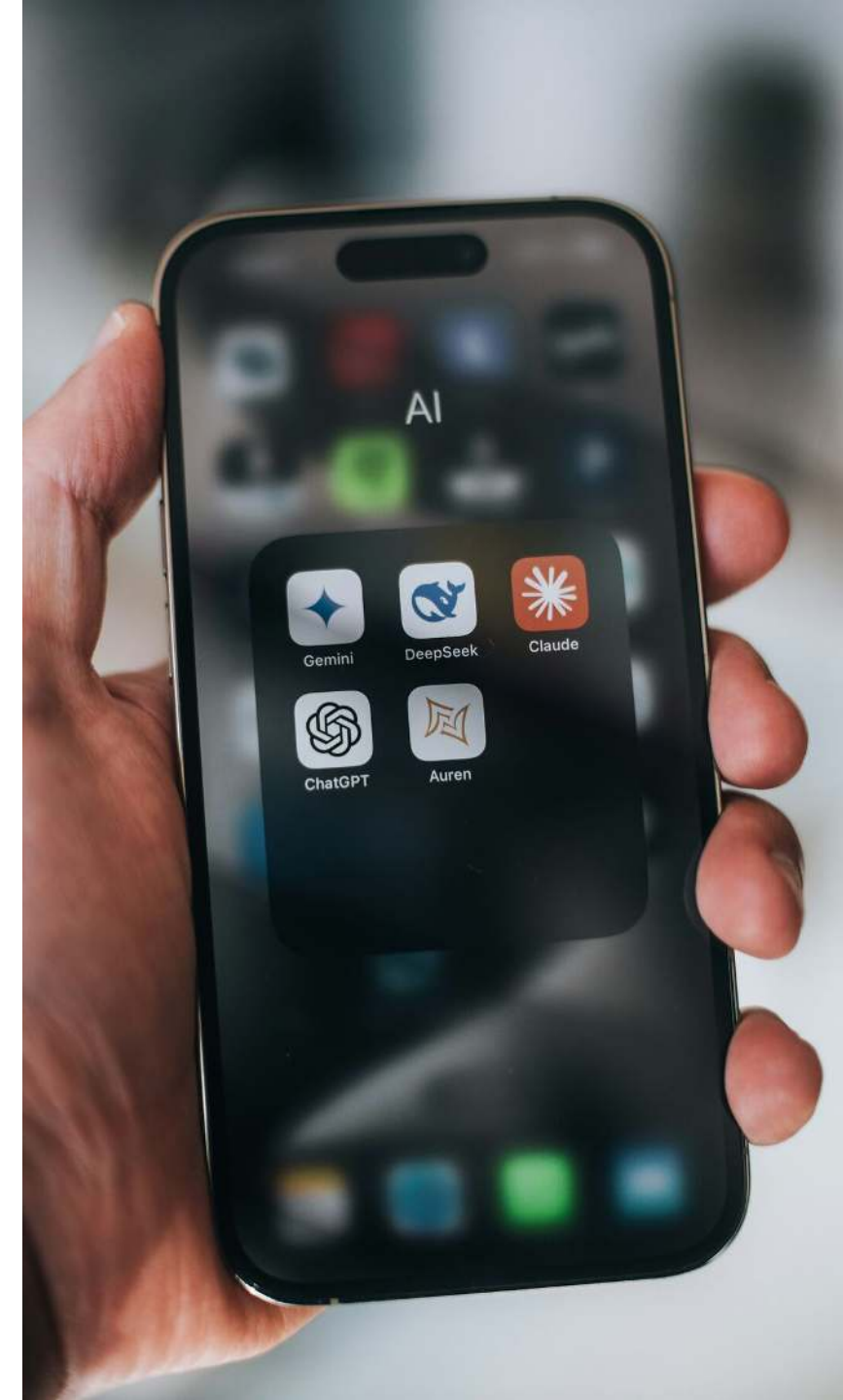
At the end of the Menti, you can add your questions for the Q&A session.

Why Test Software?



Automated Testing in the AI Era

- AI generates code; you own the consequences.
- Co-pilots don't carry pagers. You do.
- AI is the accelerator. Your test suite is the brakes. You need both to go fast.
- AI can write your tests, but it can't understand your intent. A test without intent is just a green checkmark.
- If you don't understand why a test passes, you won't understand why it fails at 3 AM.



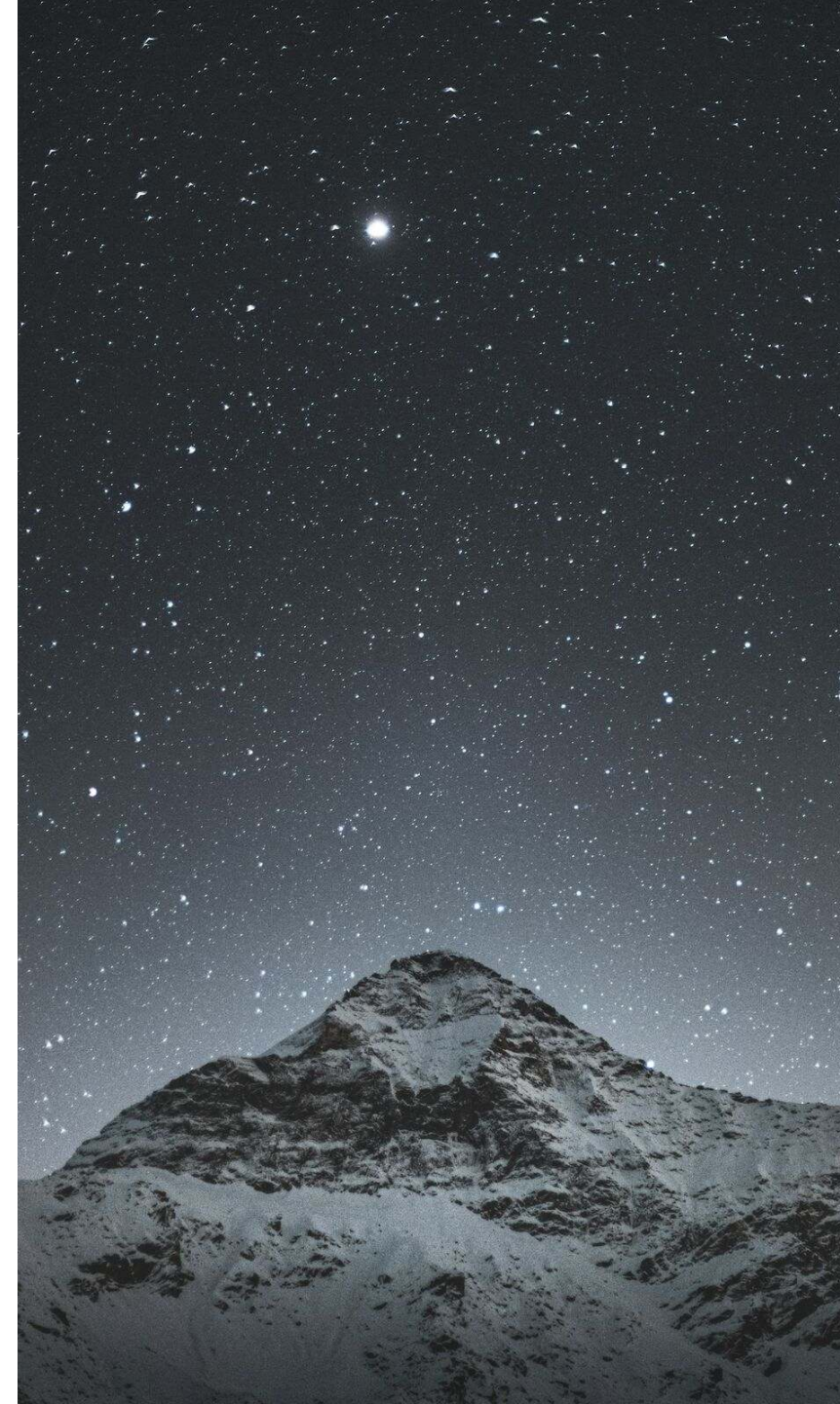
My Overall Northstar for Automated Testing

Imagine seeing this pull request on a Friday afternoon:



How confident are you to merge this major Spring Boot upgrade and deploy it to production once the pipeline turns green?

Good tests don't just catch bugs - they give you **fast feedback** and **confident deployments**.



The Hero's Journey aka. Our Agenda



Goals For This Talk

- **Provide a clear mental map** for choosing between unit, slice, and integration tests so developers stop guessing which tool to use
- **Equip attendees with practical techniques** to speed up test suites and validate test quality
- **Build confidence to refactor fearlessly** by creating tests that catch real bugs, not just achieve coverage metrics



About Philip

- Self-employed developer from Herzogenaurach, Germany (Bavaria) 🍺
- Blogging & content creation with a focus on testing Java and specifically Spring Boot applications 🌿
- Founder of PragmaTech GmbH - Enabling Developers to Frequently Deliver Software with More Confidence 🚀
- Enjoys writing tests (sometimes even more than production code) 🧪



Act 1: The Grand Entrance

Testing Spring Boot applications can feel like entering a labyrinth blindfolded:

- Copying test configuration from AI/StackOverflow hoping it works
- The paradox of choice: `@SpringBootTest` , `@WebMvcTest` , `@DataJpaTest` , `@MockBean` , etc.
- Fear of refactoring because tests break for the wrong reasons
- Tests written to satisfy coverage metrics, not increase productivity, confidence or catch bugs



Quest 1

The Unit Testing Guardian

The Swift Gatekeeper - Blocks Those Who Overcomplicate



Our Foundation: Spring Boot Starter Test

- The "Testing Swiss Army Knife"

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-test</artifactId>
4   <scope>test</scope>
5 </dependency>
```

- Batteries-included for testing by transitively including popular testing libraries
- Out-of-the-box dependency management to ensure compatibility




```
1 ./mvnw dependency:tree
2 [INFO] ...
3 [INFO] +- org.springframework.boot:spring-boot-starter-test:jar:4.0.2:test
4 [INFO] | +- org.springframework.boot:spring-boot-test:jar:4.0.2:test
5 [INFO] | +- org.springframework.boot:spring-boot-test-autoconfigure:jar:4.0.2:test
6 [INFO] | +- com.jayway.jsonpath:json-path:jar:2.10.0:test
7 [INFO] | | \- org.slf4j:slf4j-api:jar:2.0.17:compile
8 [INFO] | +- jakarta.xml.bind:jakarta.xml.bind-api:jar:4.0.4:test
9 [INFO] | | \- jakarta.activation:jakarta.activation-api:jar:2.1.4:test
10 [INFO] | +- net.minidev:json-smart:jar:2.6.0:test
11 [INFO] | | \- net.minidev:accessors-smart:jar:2.6.0:test
12 [INFO] | | \- org.ow2.asm:asm:jar:9.7.1:test
13 [INFO] | +- org.assertj:assertj-core:jar:3.27.6:test
14 [INFO] | | \- net.bytebuddy:byte-buddy:jar:1.17.8:test
15 [INFO] | +- org.awaitility:awaitility:jar:4.3.0:test
16 [INFO] | +- org.hamcrest:hamcrest:jar:3.0:test
17 [INFO] | +- org.junit.jupiter:junit-jupiter:jar:6.0.2:test
18 [INFO] | | +- org.junit.jupiter:junit-jupiter-api:jar:6.0.2:test
19 [INFO] | | | +- org.opentest4j:opentest4j:jar:1.3.0:test
20 [INFO] | | | +- org.junit.platform:junit-platform-commons:jar:6.0.2:test
21 [INFO] | | | \- org.apiguardian:apiguardian-api:jar:1.1.2:test
22 [INFO] | | +- org.junit.jupiter:junit-jupiter-params:jar:6.0.2:test
23 [INFO] | | \- org.junit.jupiter:junit-jupiter-engine:jar:6.0.2:test
24 [INFO] | | \- org.junit.platform:junit-platform-engine:jar:6.0.2:test
25 [INFO] | +- org.mockito:mockito-core:jar:5.5.0:test
26 [INFO] | | +- net.bytebuddy:byte-buddy-agent:jar:1.17.8:test
27 [INFO] | | \- org.objenesis:objenesis:jar:3.3:test
28 [INFO] | +- org.mockito:mockito-junit-jupiter:jar:5.5.0:test
29 [INFO] | +- org.skyscreamer:jsonassert:jar:1.5.3:test
30 [INFO] | | \- com.vaadin.external.google:android-json:jar:0.0.20131108.vaadin1:test
31 [INFO] | +- org.springframework:spring-core:jar:7.0.3:compile
32 [INFO] | | +- commons-logging:commons-logging:jar:1.3.5:compile
33 [INFO] | | \- org.jspecify:jspecify:jar:1.0.0:compile
34 [INFO] | +- org.springframework:spring-test:jar:7.0.3:test
35 [INFO] | \- org.xmlunit:xmlunit-core:jar:2.10.4:test
```



What's Inside the Testing Swiss Army Knife?

- **JUnit** (currently 5, later 6): Java's de-facto standard testing framework and foundation.
- **Mockito**: Creating mock objects to simulate dependencies and verify interactions.
- **AssertJ**: Provides fluent, chainable, and readable assertions.
- **Hamcrest**: Offers flexible matchers for creating custom assertions.
- **JSONAssert**: Compares JSON strings with flexible matching options.
- **JsonPath**: Extracts and queries data from JSON similar to XPath.
- **XMLUnit**: Compares and validates XML documents.
- **Awaitility**: Handles asynchronous testing with fluent conditions.



Unit Testing Java/Spring Boot Applications 101

- **Core Concept:** Test individual components in isolation from dependencies — one unit of work at a time.
- **Confidence Gained:** Fast, high-volume verification that the smallest building blocks behave correctly under various conditions.
- **Pitfall:** Poor class design leads to untestable god classes. Good tests start with good design.
- **Tools:** JUnit, Mockito, AssertJ (or Spock, TestNG, Hamcrest).



Unit Testing has Limits

Consider this sample REST controller, what could we verify with a unit test?

```
1 @RestController
2 @RequestMapping("/api/customers")
3 public class CustomerController {
4
5     private final CustomerService customerService;
6
7     public CustomerController(CustomerService customerService) {
8         this.customerService = customerService;
9     }
10
11     @PostMapping
12     public ResponseEntity<Void> createNewCustomer(@Validated CustomerCreationRequest payload, UriComponentsBuilder uriBuilder) {
13
14         String customerId = customerService.createNewCustomer(payload.firstName());
15
16         UriComponents uriComponents = uriBuilder
17             .path("/api/customers/{id}")
18             .buildAndExpand(customerId);
19
20         return ResponseEntity.created(uriComponents.toUri()).build();
21     }
22 }
```




```
1 @ExtendWith(MockitoExtension.class)
2 class CustomerControllerUnitTests {
3
4     @Mock
5     private CustomerService customerService;
6
7     @InjectMocks
8     private CustomerController customerController;
9
10    @Test
11    void shouldCreateCustomerWhenPayloadRequestIsValid() {
12        when(customerService.createNewCustomer(anyString()))
13            .thenReturn("42");
14
15        ResponseEntity<Void> result = customerController.createNewCustomer(
16            new CustomerCreationRequest("Java", "Duke", "duke@spring.io"),
17            UriComponentsBuilder.newInstance()
18        );
19
20        assertThat(result.getStatusCode().value())
21            .isEqualTo(201);
22        assertThat(result.getHeaders().getLocation().toString())
23            .isEqualTo("/api/customers/42");
24    }
25 }
```



Things We Can't Cover With a Unit Test #0

Request Mapping: Does HTTP GET `/api/customers/{id}` actually resolve to our desired method?

```
1 @Test
2 void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4     // ...
5
6     ResponseEntity<Void> result = customerController.createNewCustomer(
7         new CustomerCreationRequest("Java", "Duke", "duke@spring.io"),
8         UriComponentsBuilder.newInstance()
9     );
10 }
```



Things We Can't Cover With a Unit Test #1

Validation: Will an incomplete request body result in a 400 bad request or return an accidental 201?

```
1 @Test
2 void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4     // ...
5
6     ResponseEntity<Void> result = customerController.createNewCustomer(
7         new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),
8         UriComponentsBuilder.newInstance()
9     );
10 }
```



Things We Can't Cover With a Unit Test #2

Serialization: Are we JSON objects serialized and deserialized correctly?

```
1 ResponseEntity<Void> result = customerController.createNewCustomer(  
2     new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),  
3     UriComponentsBuilder.newInstance());
```

```
1 {  
2     "first-name": "Java",  
3     "last_Name": "Duke",  
4     "email": "duke@spring.io"  
5 }
```



Things We Can't Cover With a Unit Test #3

Security: Are we Spring Security configuration and other authorization checks enforced?

```
1 @Test
2 void shouldCreateCustomerWhenPayloadRequestIsValid() {
3
4     // ...
5
6     ResponseEntity<Void> result = customerController.createNewCustomer(
7         new CustomerCreationRequest("Java", "Duke", "NOT_AN_EMAIL"),
8         UriComponentsBuilder.newInstance()
9     );
10 }
```





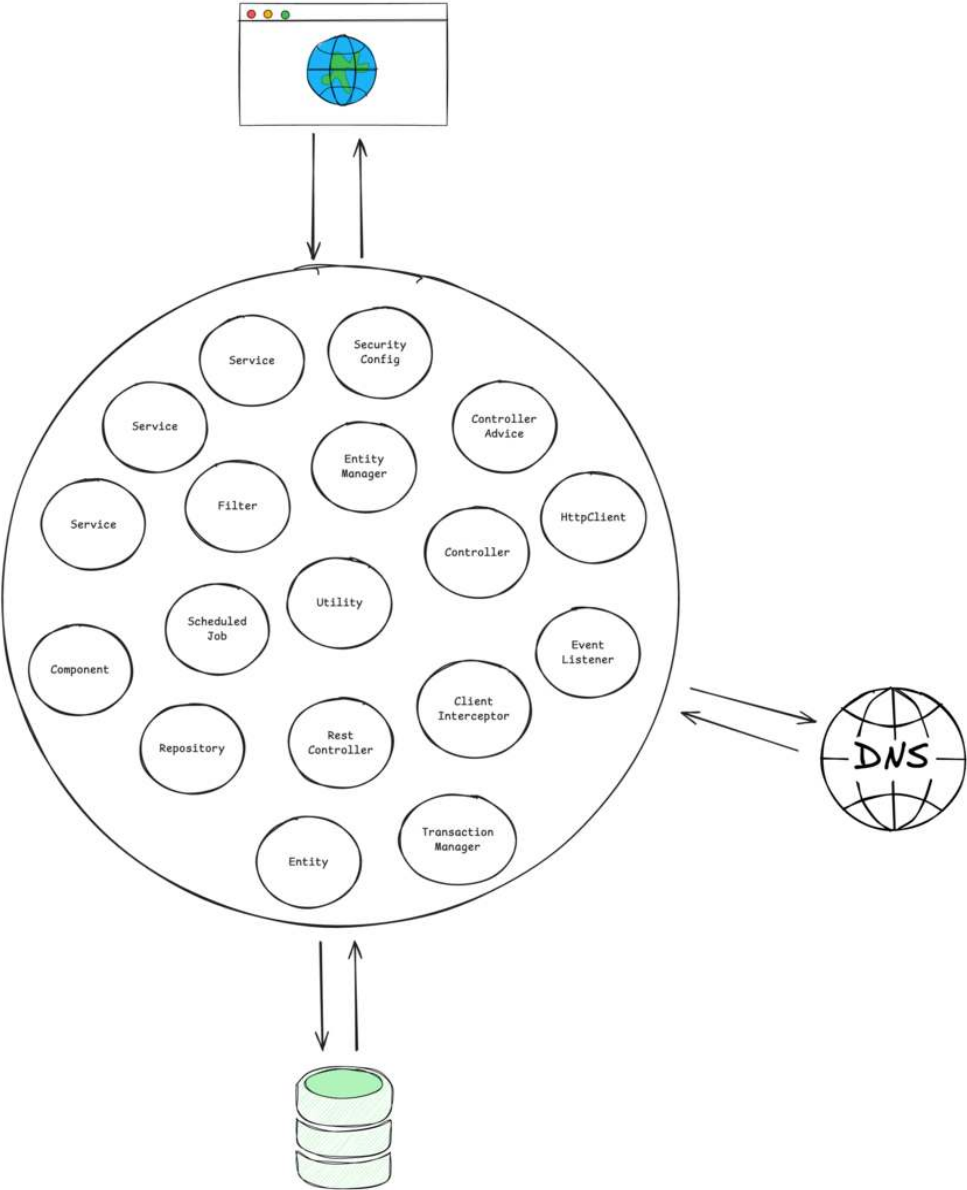
Quest 2

The Slice Testing Hydra

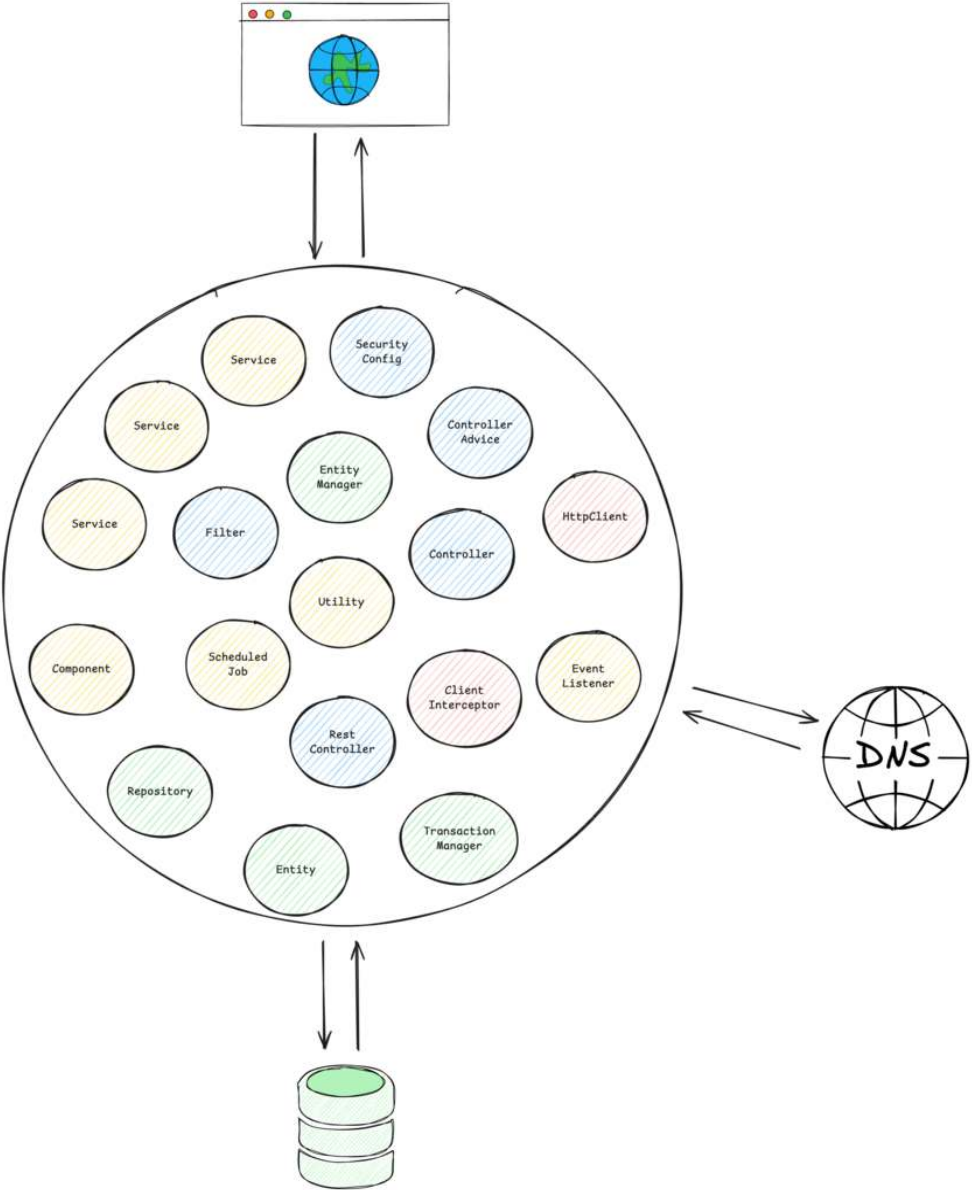
Multiple Heads, Each Guarding a Layer

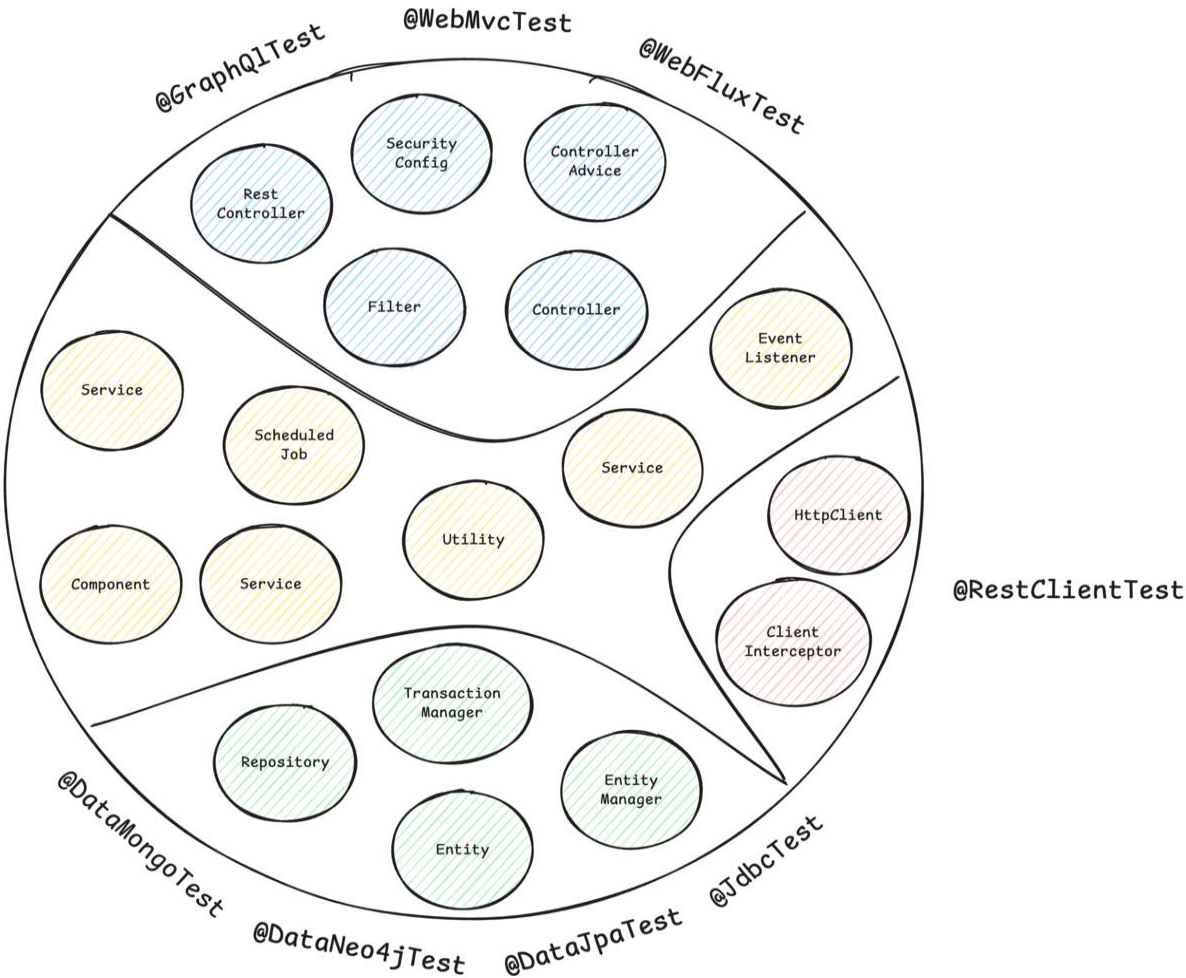


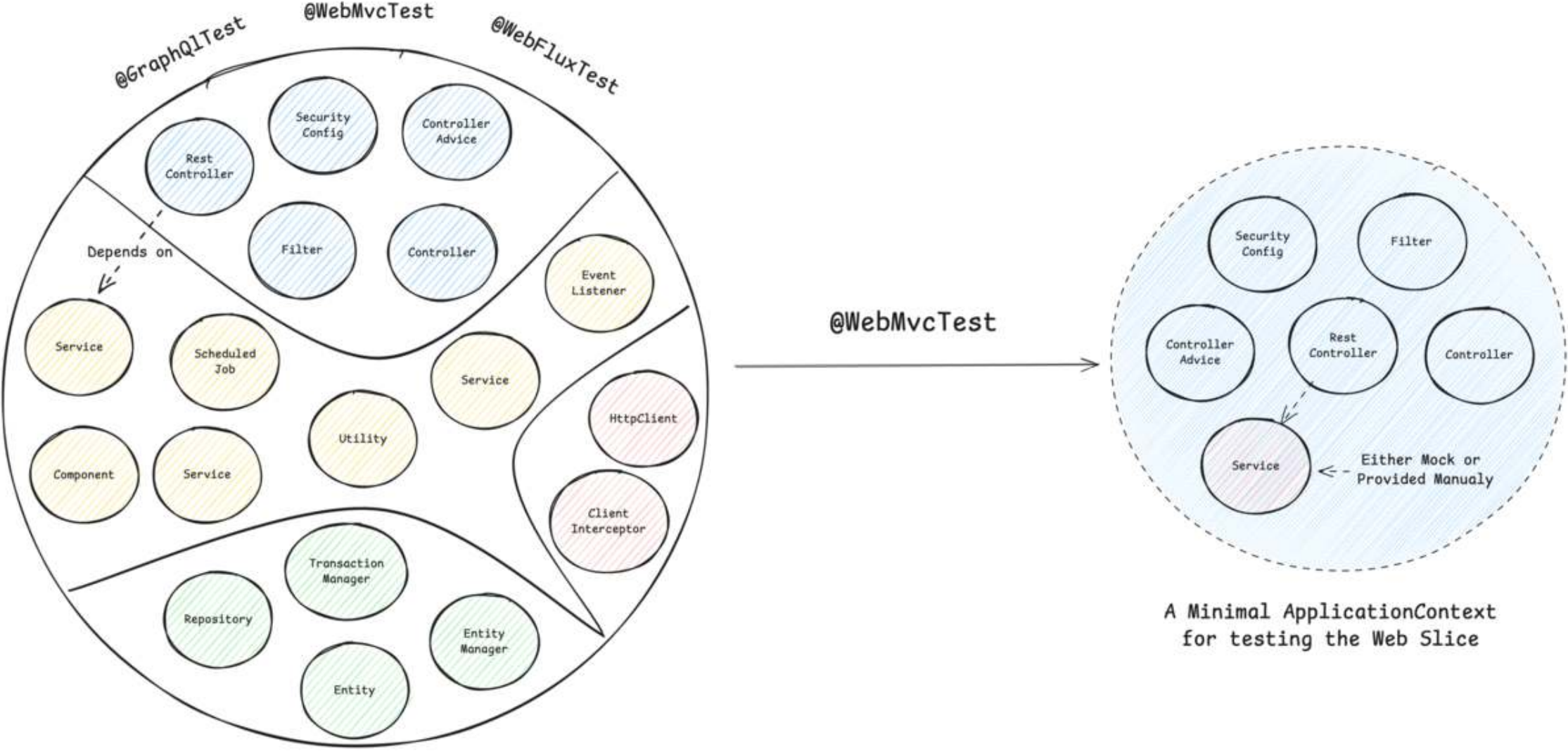
A Typical
Spring ApplicationContext
with different
Spring Beans



A Typical
Spring ApplicationContext







A Minimal ApplicationContext
for testing the Web Slice



Spring Boot Test Slice Example: @WebMvcTest

```
1 @WebMvcTest(CustomerController.class)
2 @Import(SecurityConfig.class)
3 class CustomerControllerTest {
4
5     @Autowired
6     private MockMvc mockMvc;
7
8     @MockitoBean
9     private CustomerService customerService;
10
11     @Test
12     @WithMockUser
13     void shouldReturnLocationOfNewlyCreatedCustomer() throws Exception {
14         // ...
15     }
16 }
```



Common Test Slices

- `@WebMvcTest` / `@WebFluxTest` - Controller layer
- `@DataJpaTest` / `@JdbcTest` - Persistence layer
- `@JsonTest` - JSON serialization/deserialization
- `@RestClientTest` - RestTemplate testing
- etc.



@DataCouchbaseTest
@DataLdapTest
@RestClientTest
@JsonTest
@JooqTest
@WebFluxTest
@DataRedisTest
@WebMvcTest
@DataMongoTest
@DataCassandraTest
@GraphQLTest
@DataNeo4jTest
@WriteYourOwn*
@DataJpaTest
@SqsTest
@JdbcTest
@DataElasticsearchTest



Sliced Testing Spring Boot Applications 101

- **Core Concept:** Test a specific "slice" or layer of your application by loading a minimal, relevant part of the Spring `ApplicationContext`.
- **Confidence Gained:** Helps validate parts of your application where pure unit testing is insufficient, like the web, messaging, or data layer.
- **Prominent Examples:** Web layer (`@WebMvcTest`) and database layer (`@DataJpaTest`)
- **Pitfalls:** Requires careful configuration to ensure only the necessary slice of the context is loaded.
- **Tools:** JUnit, Mockito, Spring Test, Spring Boot, Testcontainers



Quest 3

The Integration Testing Dragon

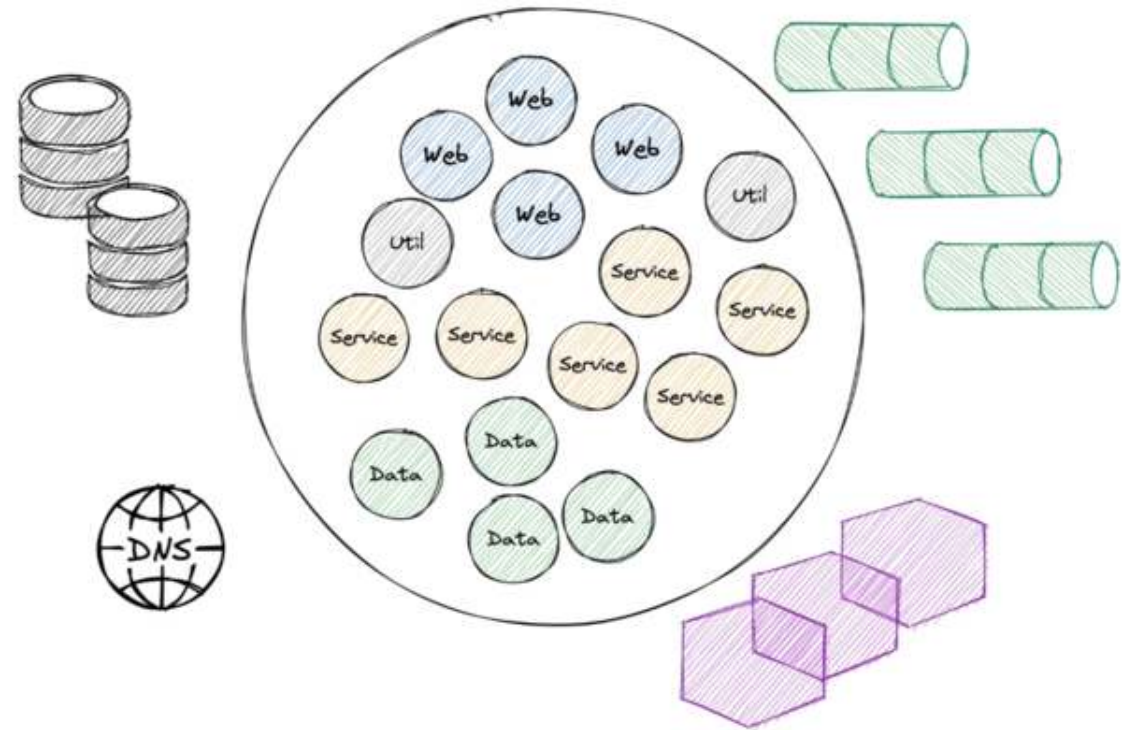
Guards the Full Treasure - but Demands
Patience



```
@SpringBootTest
class ApplicationTest {

    @Test
    void contextLoads() {
    }

}
```



Starting the Entire **ApplicationContext**

- **Problem #1:** How to ensure surrounding infrastructure (e.g. database, queues, etc.) is present?
- **Problem #2:** How to handle HTTP communication from our application to remote services?
- **Problem #3:** How to keep our build time at a reasonable duration?



Provide External Infrastructure with Testcontainers (Problem #1)

Running infrastructure components (databases, message brokers, etc.) in Docker containers for our tests becomes a breeze with [Testcontainers](#):

```
1 @Container
2 @ServiceConnection
3 static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:16-alpine")
4     .withDatabaseName("testdb")
5     .withUsername("test")
6     .withPassword("test")
7     .withInitScript("init-postgres.sql");
```

This gives us an ephemeral PostgreSQL database for our tests:

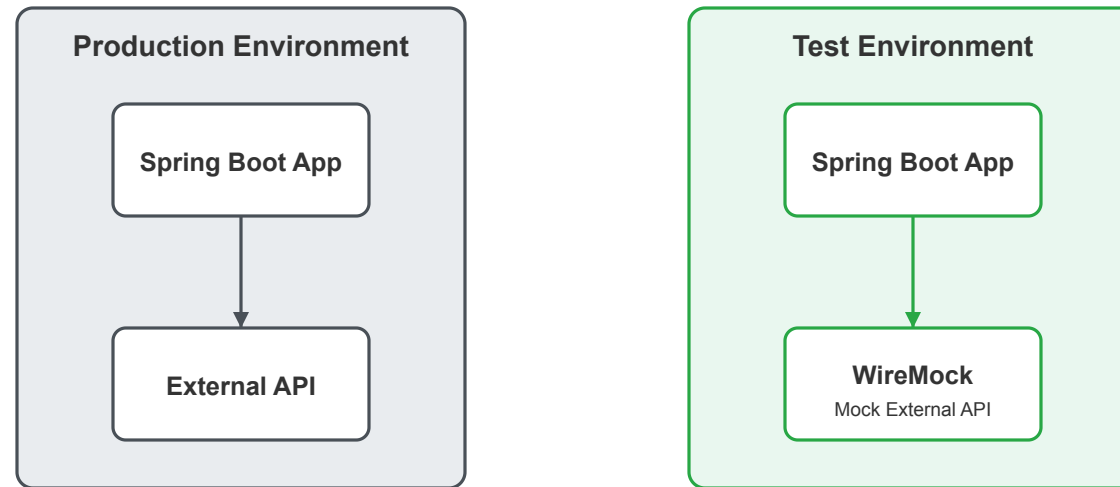
```
1 $ docker ps
2 CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS          PORTS                               NAMES
3 a958ee2887c6   postgres:16-alpine                 "docker-entrypoint.s..." 10 seconds ago   Up 9 seconds    0.0.0.0:32776->5432/tcp, [::]:32776->5432/tcp affectionate_cannon
4 ad0f804068dc   testcontainers/ryuk:0.12.0         "/bin/ryuk"              10 seconds ago   Up 9 seconds    0.0.0.0:32775->8080/tcp, [::]:32775->8080/tcp testcontainers-ryuk-1f9f76a6-46d4-4e19-85c1-e8364da12804
```



Stub External HTTP Services with WireMock (Problem #2)

Consider [WireMock](#) to stub external HTTP services during tests.

WireMock: HTTP Service Stubbing for Testing



Key Benefits of WireMock

- Controlled test environment
- No network dependency
- Predictable responses
- Simulate error conditions
- Fast test execution
- Java integration



Using WireMock for Integration Tests

- Run as in-memory service or Docker container to simulate connected HTTP services
- Override HTTP clients to connect to the WireMock server during tests

```
1 TestPropertyValues.of(  
2     "clients.open-library.base-url=http://localhost:" + wireMockServer.port())  
3     .applyTo(applicationContext);
```

```
1 wireMockServer.stubFor(  
2     get(urlPathEqualTo("/api/books/" + isbn))  
3         .willReturn(aResponse()  
4             .withHeader(HttpHeaders.CONTENT_TYPE, MediaType.APPLICATION_JSON_VALUE)  
5             .withBodyFile("book-response-success.json"))  
6 );
```



Starting the Entire Spring Context - Version 1

- The test and the context run in the same thread, hence we can rollback with `@Transactional` and simply override the security context with `@WithMockUser`

```
1 @SpringBootTest
2 // which is @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.MOCK)
3 @AutoConfigureMockMvc
4 class ApplicationMockWebIT {
5
6     // @LocalServerPort
7     // private int port; <-- this would fail the test, there is no local port occupied
8
9     @Test
10    @WithMockUser
11    void givenCustomersThenReturnListForAuthenticatedUser(@Autowired MockMvc mockMvc) throws Exception {
12        mockMvc
13            .perform(get("/api/customers")
14                .header(ACCEPT, APPLICATION_JSON))
15            .andExpect(status().is(200))
16            .andExpect(content().contentType(APPLICATION_JSON))
17            .andExpect(jsonPath("$.size()", is(1)));
18    }
19 }
```



Starting the Entire Spring Context - Version 2

- We access the application over HTTP like a user, the test and context run in separate threads (no `@Transactional` rollback), requires HTTP authentication

```
1 @AutoConfigureWebTestClient // required since Spring Boot 4
2 @SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
3 class ApplicationServletContainerIT {
4
5     @LocalServerPort
6     private int port; // <-- we're running on a real port
7
8     @Test
9     void contextLoads(@Autowired WebTestClient webTestClient) {
10         webTestClient
11             .get()
12             .uri("/api/customers")
13             .header("Authorization", "Basic " + Base64.getEncoder().encodeToString("user:dummy".getBytes()))
14             .exchange()
15             .expectStatus()
16             .isOk();
17     }
18 }
```



Integration Testing Spring Boot Applications 101

- **Core Concept:** Start the entire Spring application context, often on a random local port, and test the application through its external interfaces (e.g., REST API).
- **Confidence Gained:** Validates the integration of all internal components working together as a complete application.
- **Best Practices:** Use `@SpringBootTest` to run the app on a local port.
- **Pitfalls:** Slower to run than unit or sliced tests. Managing the lifecycle of dependent services can be complex.
- **Tools:** JUnit, Mockito, Spring Test, Spring Boot, Testcontainers, WireMock (for mocking external HTTP services), Selenium (for browser-based UI testing)



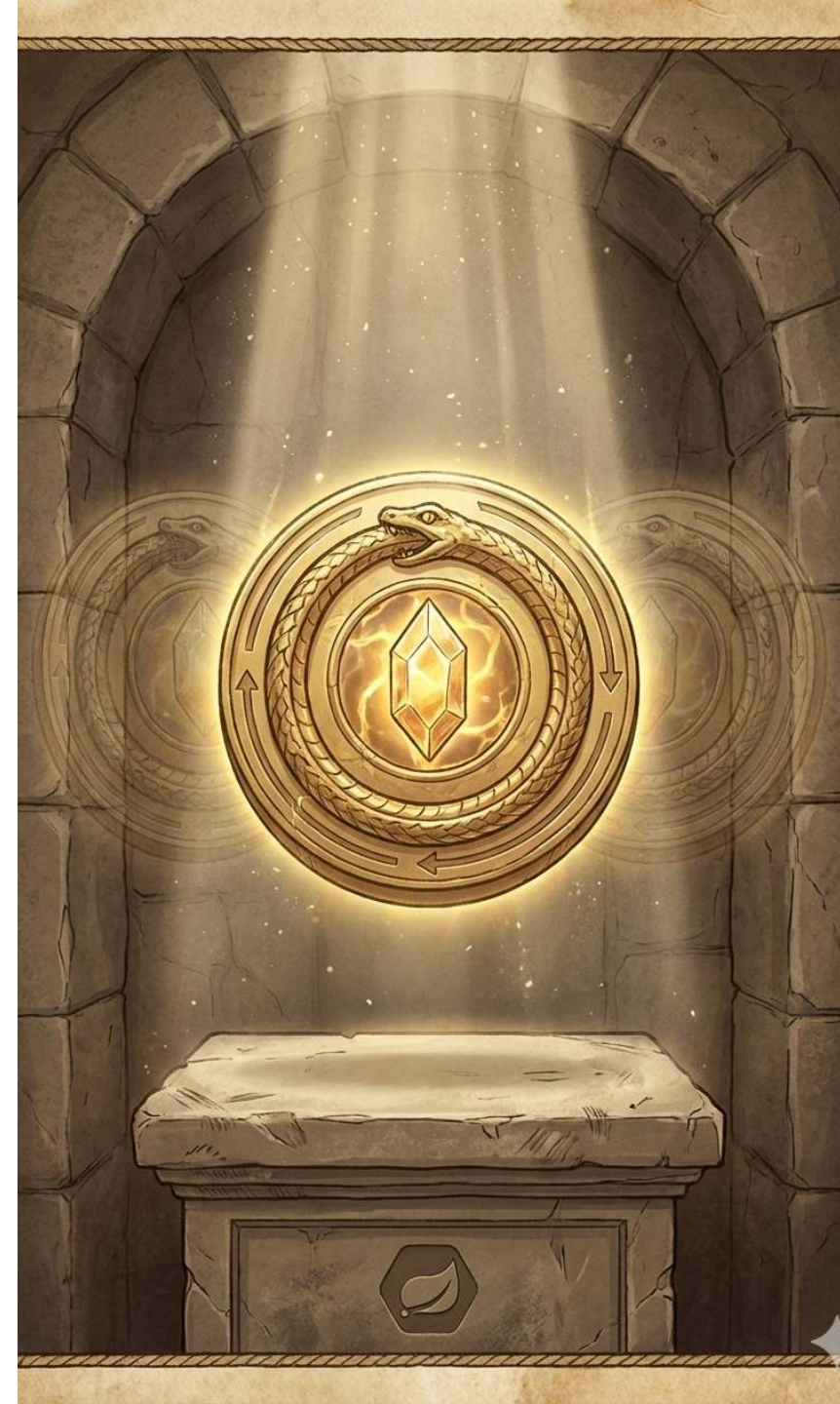
... but what about **Problem #3**: How to keep our build time at a reasonable duration?



Quest Item 1

The Caching Amulet

Helps You Reuse What You Already Built



Integration Testing - The Need for Speed

- **The Problem:** Integration tests require a started & initialized Spring `ApplicationContext`, which slows down the build
- **The Solution:** Spring Test `TestContext` Caching – stores an already started Spring `ApplicationContext` for reuse
- This feature is part of Spring Test (included in every Spring Boot project via `spring-boot-starter-test`)

Example of speed improvement:



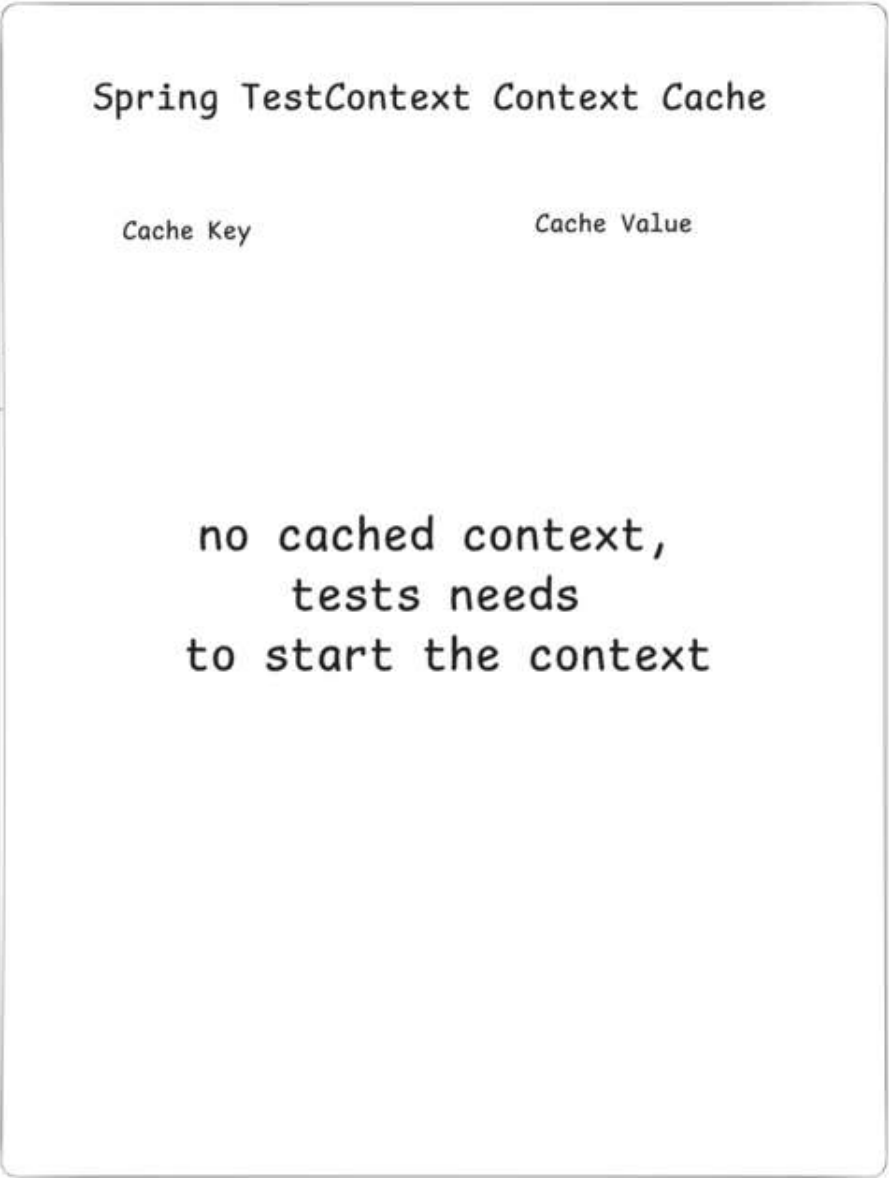
Test run time: 3506ms

OrderIT
requiring context
configuration with the
hash code #327321289

PaymentIT
requiring context
configuration with the
hash code #327321289

CheckoutIT
requiring context
configuration with the
hash code #565212314

Context cache lookup



Test run time: 3506ms

OrderIT
requiring context
configuration with the
hash code #327321289

Test run time: 406ms

PaymentIT
requiring context
configuration with the
hash code #327321289

CheckoutIT
requiring context
configuration with the
hash code #565212314

test gets cached context and
finishes the test very fast

Spring TestContext Context Cache

Cache Key

Cache Value

#327321289



Test run time: 3506ms

OrderIT
requiring context
configuration with the
hash code #327321289

Test run time: 406ms

PaymentIT
requiring context
configuration with the
hash code #327321289

Test run time: 4506ms

CheckoutIT
requiring context
configuration with the
hash code #565212314

Spring TestContext Context Cache

Cache Key

Cache Value

#327321289



#565212314



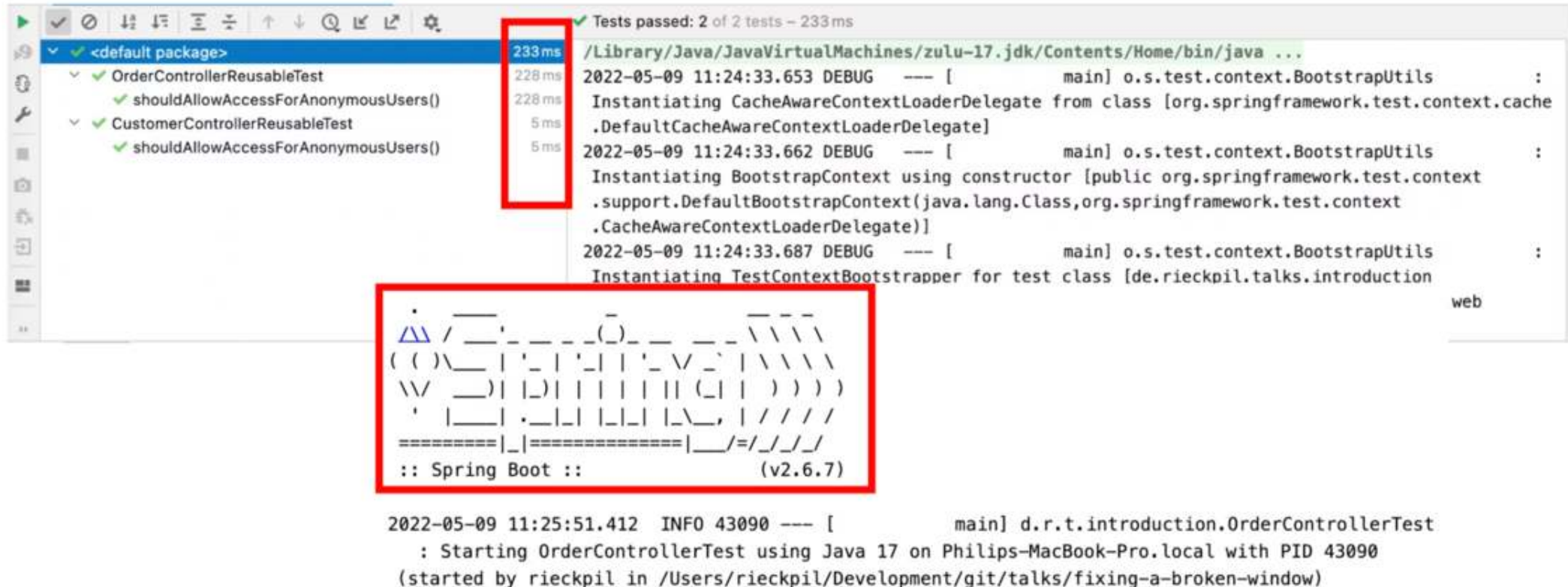
How the Cache Key is Built

```
1 // DefaultContextCache.java
2 private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
3     Collections.synchronizedMap(new LruCache(32, 0.75f));
```

The following information is part of the Cache Key (`MergedContextConfiguration`):

- `activeProfiles ( @ActiveProfiles )`
- `contextInitializersClasses ( @ContextConfiguration )`
- `propertySourceLocations ( @TestPropertySource )`
- `propertySourceProperties ( @TestPropertySource )`
- `contextCustomizer ( @MockitoBean , @MockBean , @DynamicPropertySource , ...)`
- etc.





Detect Context Restarts - with Logs

```
<logger name="org.springframework.test.context" level="TRACE" />
```

```
2022-05-05 14:27:38.136 DEBUG 42500 --- [          main] org.springframework.test.context.cache : Spring test
ApplicationContext cache statistics: [DefaultContextCache@68e62b3b size = 1, maxSize = 32, parentContextCount = 0,
hitCount = 11, missCount = 1]
2022-05-05 14:27:38.136 DEBUG 42500 --- [          main] tractDirtiesContextTestExecutionListener : After test class:
context [DefaultTestContext@325bb9a6 testClass = ApplicationIT, testInstance = [null], testMethod = [null], testException
= [null], mergedContextConfiguration = [WebMergedContextConfiguration@1d12b024 testClass = ApplicationIT, locations =
'{}', classes = '{class de.rieckpil.talks.Application}', contextInitializerClasses = '[]', activeProfiles = '{}',
propertySourceLocations = '{}', propertySourceProperties = '{org.springframework.boot.test.context
.SpringBootTestContextBootstrapper=true, server.port=0}', contextCustomizers = set[org.springframework.boot.test.context
.filter.ExcludeFilterContextCustomizer@5215cd9a, org.springframework.boot.test.json
.DuplicateJsonObjectContextCustomizerFactory$DuplicateJsonObjectContextCustomizer@9257031, org.springframework.boot.test
```



Detect Context Restarts - with Tooling



An [open-source Spring Test utility](#) that provides visualization and insights for Spring Test execution, with a focus on Spring context caching statistics.

Overall goal: Identify optimization opportunities in your Spring Test suite to speed up your builds and ship to production faster and with more confidence.



The Final Boss

Developers tend to consult AI/StackOverflow for integration test issues and often copy advice from the internet without knowing the implications:

```
1 @SpringBootTest
2 @DirtiesContext
3 // this instructs Spring to remove the context from the cache
4 // and rebuild a new context on every request
5 public abstract class AbstractIntegrationTest {
6
7 }
```

The setup above will **disable** the context caching feature and slow down the builds significantly!



New in Spring Framework 7: Pausing Contexts

See Release Notes von [Spring Framework 7](#).

Pausing of Test Application Contexts

The Spring TestContext framework is caching application context instances within test suites for faster runs. As of Spring Framework 7.0, we now pause test application contexts when they're not used.

This means an application context stored in the context cache will be stopped when it is no longer actively in use and automatically restarted the next time the context is retrieved from the cache.

Specifically, the latter will restart all auto-startup beans in the application context, effectively restoring the lifecycle state.



Quest Item 2

The Lightning Shield

Many Cores, One Goal



Test Parallelization

Goal: Reduce build time and get faster feedback

Requirements:

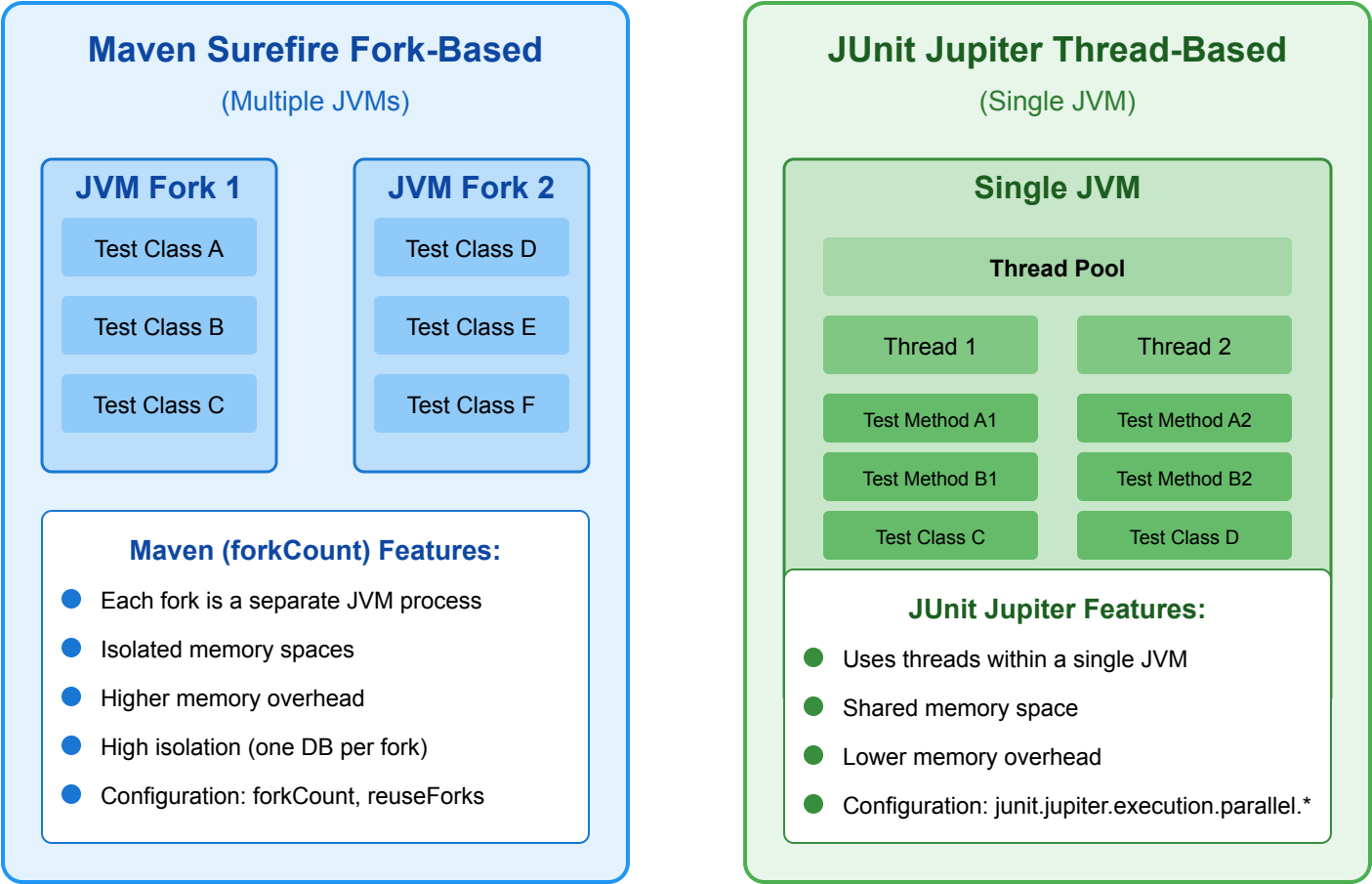
- No shared state
- No dependency between tests and their execution order
- No mutation of global state

Two ways to achieve this:

- Fork a new JVM with Surefire/Failsafe (or for the Gradle test task) and let it run in parallel
- Use JUnit Jupiter's parallelization mode and let it run in the same JVM with multiple threads



Java Test Parallelization Options



Reuse Containers with Testcontainers

- Bootstrapping new containers can take a significant amount of time
- Enable container reuse in Testcontainers when possible: `.withReuse(true)`
- Singleton containers per test run are preferable to `@Testcontainers` (container per test class)
- Speed up container startup with e.g. predefined database snapshots

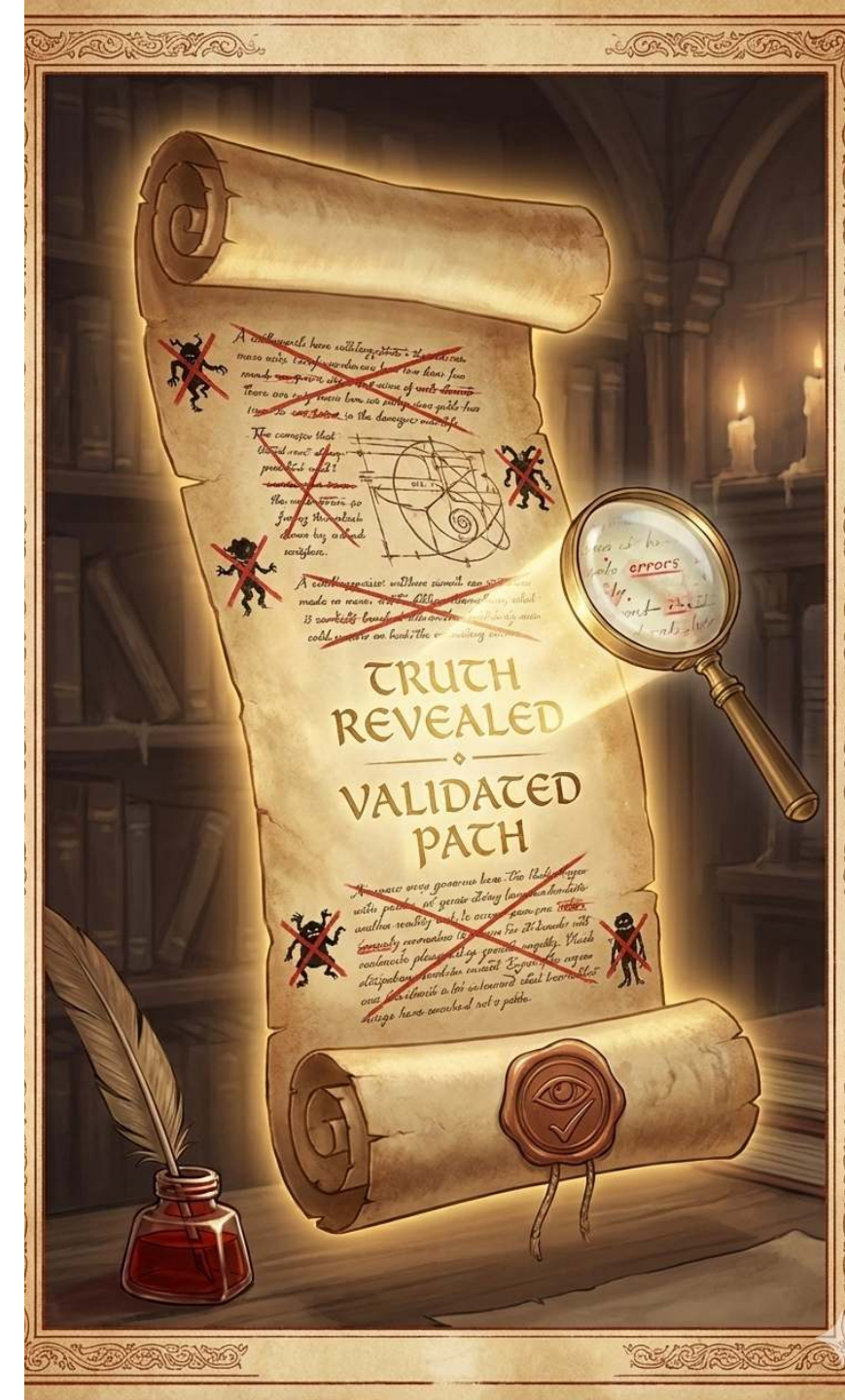
```
1 private static PostgreSQLContainer<?> postgresModule = new PostgreSQLContainer<>("myteampostgres:42")
2   .withDatabaseName("testdb")
3   .withUsername("testuser")
4   .withPassword("testpass");
5
6 static {
7     postgresModule.start();
8 }
```



Quest Item 3

The Scroll of Truth

Coverage Lies, Mutants Don't



Introducing: Mutation Testing

- Having high code coverage might give you a **false sense of security**
- Mutation Testing with **PIT**
- Beyond Line Coverage: Traditional tools like JaCoCo show which code runs during tests, but PIT verifies if our tests actually detect when code behaves incorrectly by introducing "**mutations**" to our source code.
- Quality Guarantee: PIT automatically **modifies our code** (changing conditionals, return values, etc.) to ensure our tests fail when they should, **revealing blind spots** in seemingly comprehensive test suites.



```
public Long registerUser(String username, int age) { no usages

    if(this.userRepository.findByUsername(username).isPresent()) {
        throw new IllegalArgumentException("Username already exists");
    }

    if(age <= 18) {
        throw new IllegalArgumentException("User is underaged");
    }

    if(username.equals("ADMIN")) {
        // default admin user id
        return 3L;
    }

    return userRepository.save(username);
}
```



Existing Code & Tests

Mutant from UserService
(e.g. <= gets !=)



run relevant tests



Test fails X

Mutant was successfully detected :)

Mutant from UserService
(e.g. equals gets !=equals)



run relevant tests



Test is green

Mutant wasn't detected :(

Mutant from UserService
(e.g. return 3 gets retun 42)



run relevant tests



Test is green

Mutant wasn't detected :(



Act 5: The Triumphant Exit

- Spring Boot applications come with batteries-included and excellent testing support
- We've completed three main quests: Unit testing, Sliced testing, and Integration testing
- Three core quest items help us to speed up and validate our tests:
 - Context Caching
 - Test Parallelization
 - Mutation Testing



Spring Boot 4 - Testing Support Keeps Improving

- **RestTestClient:** Modern, fluent alternative for the `TestRestTemplate` / `WebTestClient` / `RestAssured`.
- **Context pausing:** Cached test contexts are now automatically paused, eliminating resource conflicts from background processes.
- **JUnit 6:** Drop-in upgrade from JUnit 5 - far smoother than the JUnit 4 → 5 migration.
- **Testcontainers 2.0:** New `testcontainers-` prefix for modules, JUnit 4 support removed.
- **Bean overrides for non-singletons:** `@MockitoBean` and `@TestBean` now work with prototype and custom-scoped beans.



What's Next?

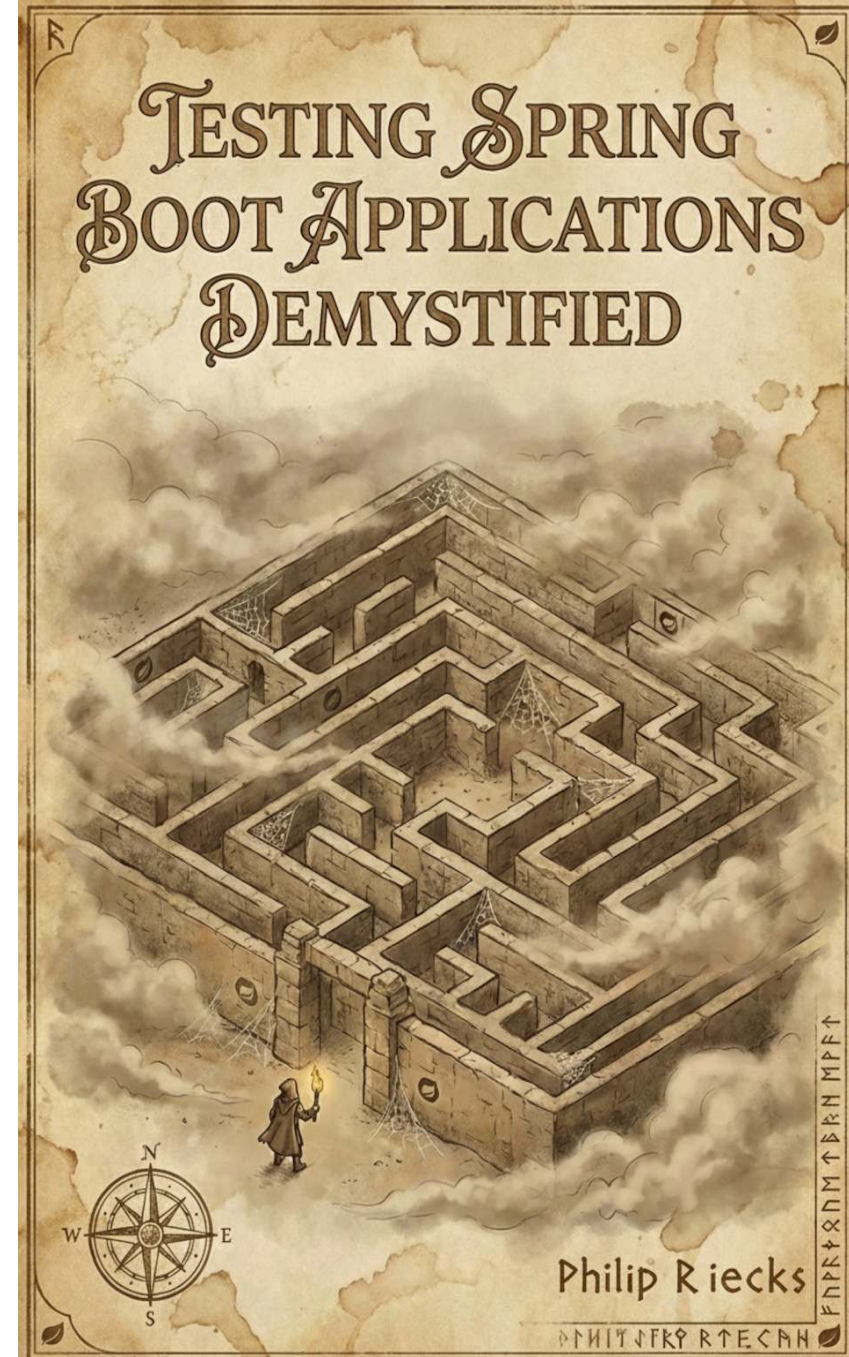
Testing is a team sport, make sure your whole team levels up together

- Spring Boot [testing workshops](#) (in-house/remote/hybrid)
- Further Spring Boot testing resources (courses, eBooks, articles) at rieckpil.de
- [Consulting offerings](#), e.g. the Test Maturity Assessment for projects/teams



Don't Leave Empty-Handed

- Get the complementary **Testing Spring Boot Applications Demystified** for free (instead of \$9)
- 120+ Pages with practical hands-on advice to ship code with confidence
- Get the eBook by joining our [newsletter](#)



Joyful Testing!

Get your Spring Boot testing eBook:



Now it's time for Q&A! If you have any questions feel free to ask via Zoom chat or within Menti (last question).

Reach out any time via: [LinkedIn](#) (Philip Riecks) or [Mail](mailto:philip@pragmatech.digital) (philip@pragmatech.digital)

